

Who Told the Agent What to Do? AI Instruction Files and Accountability Before Code Review*

Harin Lee[†]

harinlee0503@gmail.com
Ewha Womans University
Seoul, Republic of Korea

Donghyun Sohn*

donghyun.sohn@u.northwestern.edu
Northwestern University
Evanston, Illinois, USA

Abstract

Developers increasingly use repository files such as `CLAUDE.md`, `AGENTS.md`, and `.cursor/rules` to guide AI coding agents before they produce code. These files tell the agent what to change, what to leave alone, which part of the project to trust, and even how to describe its own work. Code review, though, usually starts later, once a patch already exists. At that point, a reviewer sees the diff and “AI-assisted” note, but not the instructions that shaped the patch. We study this gap with 4,664 such files from 4,485 public GitHub repositories. In a stratified sample of 499 pairs of an AI file and the same repository’s README, a classifier found agent-only rules and project judgments together in 88.2% of pairs, and a stricter hand check confirmed both in 57%. Only 16.7% of READMEs point to the AI file by name; even counting any hint, such as a mention of Claude, Cursor, or a coding agent, only 33.8% do. For most repositories, a person reading the README would not know the agent had its own set of rules. We argue that accountability for AI-assisted work should start before the patch, not only after it. The question is not only whether AI was used, but what the agent was told, and whether that can be reviewed, credited, and challenged. We frame this as an upstream condition, not as proof that work is already being misattributed.

CCS Concepts

• **Human-centered computing** → Empirical studies in collaborative and social computing; • **Software and its engineering** → Collaboration in software development.

1 Introduction

Good code review does more than find bugs. Prior work [2] shows that review also supports knowledge transfer, team awareness, and improved solutions. It is also a conversation about judgment. A reviewer asks why a design was chosen, why one module and not another, why not the simpler path, and the author defends the choice, changes it, or convinces the reviewer. Through this back-and-forth a team builds trust and learns what each person actually understands.

Now imagine reviewing an AI-assisted pull request. The contributor has disclosed that a coding agent was used. The diff is visible, the commit history is intact, and the provenance requirement looks satisfied. What stays out of view is that the agent was told, in a separate file, to preserve an undocumented invariant, to avoid one

architectural path, and to leave no sign that an agent helped write the commit. The disclosure records that an agent was involved. It does not show the conditions under which the contribution was formed, and it gives the reviewer little to push back on.

Disclosing that AI was used is not enough. The harder question is what the agent was told before the code was written. A patch can appear to reflect the contributor’s own judgment. Some of that reasoning may already be encoded in an agent-facing file that the reviewer never opens. The patch itself remains open to review, while the rule or judgment behind it may remain outside the review. We call this a *review gap*: the rules and judgments that shaped the work exist, but not where people normally look when they review it. This view of accountability builds on prior work [10] that emphasizes making enough of how work takes shape visible for others to question it and hold someone answerable. We extend this view to instructions that are written before the contribution itself takes shape.

The risk is also cumulative. One AI-assisted patch may pass review on its own. But if reviewers keep seeing only the diff and not the agent-facing rules behind it, small unexamined choices can build up into architecture, and a team can end up knowing what the system does but not why it became that way.

Our position is that accountability for AI-assisted work begins too late when it focuses on output disclosure alone. Instruction files let us move the question upstream, to the rules, judgments, and authorities available to the agent before a patch is produced. Prior studies and industry reports have examined these files mainly as configuration or efficiency aids [1, 3, 4, 6]. We instead examine them as accountability artifacts: records of the conditions under which agent-mediated work takes shape. Coding repositories provide a concrete setting for studying this instruction layer. The underlying accountability gap can extend beyond coding agents whenever other agents receive instructions that collaborators do not see.

We are careful about scope. We do not claim that this gap already makes a person’s own contribution impossible to separate from the agent’s, nor that every instruction is read or followed by every agent. We show an earlier condition: the context that shapes a contribution can sit outside the ordinary path through which collaborators later judge who did what.

2 What the Files Tell the Agent

We assembled 4,664 instruction files from 4,485 public GitHub repositories (collected on April 27, 2026) across six conventions: `CLAUDE.md`, `AGENTS.md`, Copilot instructions (`.github/copilot-instructions.md`), GitHub path-scoped instructions (`.github/instructions/`), Cursor rules (`.cursor/rules`), and GEMINI.md. Among repositories with classifiable contribution histories, 68.3% were

*Position paper accepted to the workshop “Nobody Did This”: Contribution, Originality, and Accountability in Agent-Mediated Collaboration [5] at the 2026 ACM Conference on Computer-Supported Cooperative Work and Social Computing (CSCW ’26), Salt Lake City, UT, USA, October 2026.

[†]Both authors contributed equally to this work.

Table 1: Empirical backbone. All numbers are defined and sourced in the text.

Measure	Value
Repositories / instruction files	4,485 / 4,664
Solo-classified repositories	68.3%
Same-repository pairs analyzed	499
Constraint + judgment co-occur (classifier)	88.2%
Same co-occurrence, strict hand-confirmed subset	57%
Human precision (constraints / judgments)	.88 / .84
READMEs naming the file (exact / any signal)	16.7% / 33.8%
Repos with a CONTRIBUTING.md (all / solo)	16.3% / 11.8%

classified as solo repositories. The files contain substantially more than build and run commands. Project-domain knowledge and architectural conventions are more common than build and run steps. In practice this is the kind of knowledge that conditions behavior: which module is the source of truth, which directory must never be edited by hand, which invariant a change must preserve, what a project-specific term means. It is the material a later reader would struggle to recover from the diff alone, and the file is designed to make it available to the agent before it writes anything. Writing these files down is itself a form of articulation work, the behind-the-scenes effort of making joint activity fit together [7, 9]. The twist is that the effort is directed at a non-human worker, and once written it becomes a new layer of the invisible work that documentation both records and hides [8].

We focus on two kinds of difference. A *behavioral constraint* is a rule about what the agent must or must not do, such as “never edit files under gen/ by hand.” A *project judgment* is a claim about what the project treats as true or authoritative, for example “module X is the source of truth for configuration.” Our central analysis pairs an instruction file with the README from the same repository. We drew a stratified sample of 500 pairs, balanced across the six conventions and across user and organization owners so that no single tool or account type dominates the estimate. We asked Google’s Gemini 3 Flash (gemini-3-flash-preview) to identify behavioral constraints and project judgments that appear in the agent-facing file but not in the README. For every flag, the model had to return a word-for-word quote from the instruction file and leave uncertain cases unflagged. One pair failed to parse, leaving 499 pairs for analysis. We then manually checked all 323 flagged items in a 60-pair validation set. We accepted a flag only when the quote genuinely fit its category and the README contained no equivalent in any wording. Borderline cases were counted as not missing, yielding conservative precision estimates of 0.88 for constraints and 0.84 for judgments. We base our central claims on these two categories and leave weaker signals, including security and conflict detection, aside. Table 1 lists the backbone numbers.

The README is a deliberately narrow first-contact baseline, not the whole of a project’s documentation. Since specialized documents naturally contain rules the README omits, divergence alone proves little. We confirmed this by running the same classification procedure on 150 CONTRIBUTING.md and README pairs; as expected, human-facing guidelines also diverge. Our core argument,

therefore, is not that agent files uniquely diverge from README content. The important difference is the role that divergence can play: agent-facing instructions are written to condition an automated contributor’s next actions. They can also introduce a new agent-facing documentation layer into repositories that do not maintain a CONTRIBUTING.md. The next section develops these two points.

3 Divergence Is Common; Surfacing Is Not

The obvious objection is correct. Specialized documents are meant to hold material a README omits. In our baseline, behavioral rules missing from the README appear in 99.3% of CONTRIBUTING.md pairs, even more often than the 94.4% for instruction files. Rule divergence on its own is not an agent-specific finding, and we do not present it as one.

Three things make the agent-facing case different. First, the role is performative as well as textual. A CONTRIBUTING.md addresses a person who may read, question, or ignore it. An instruction file is written to enter an automated contributor’s working context and condition what it does next. Second is reach. Only 16.3% of our repositories keep a CONTRIBUTING.md, and only 11.8% of solo-classified ones do, yet an instruction file adds an agent-facing documentation layer to many repositories that do not maintain a CONTRIBUTING.md. Third is the kind of content. Project judgments are more common in instruction files than in CONTRIBUTING.md files (92.8% vs. 82.7%): the agent channel pulls in reasoning, not only procedure.

Solo is not really the absence of collaboration; it is often its earliest moment. A fair objection is that witnessing requires another contributor, which a solo repository lacks. But solo is not always permanent. The instruction file written today outlives the moment it was written. It guides the agent when the first outside pull request arrives, when a second maintainer is onboarded, when the project is forked, and when the code is handed off. At each of those moments the rules and judgments that shaped earlier agent work are already fixed, in a file the newcomer is unlikely to read. The solo majority is therefore not a gap in the argument. It is where the conditions for later witnessing are set, before any witness is present.

The two documents also read differently. Across the 3,294 repositories included in this analysis, instruction files lean on directive language (command markers such as *must* and *never*, with a matched-pairs rank-biserial effect size $r = 0.83$) and imperative verbs ($r = 0.69$), while READMEs keep second-person address ($r = -0.59$) and more code examples. Here, r denotes the direction of the difference in a Wilcoxon signed-rank test, where positive values indicate the feature is more common in the instruction file. The same project is written once as instruction and once as orientation. The question is whether that split in style is matched by a split in substance.

The pattern is asymmetry, not contradiction. By the classifier, behavioral constraints and unsurfaced project judgments co-occur in 88.2% of pairs. That is an estimate; the stricter human-confirmed figure is lower. In the hand-checked validation set, 78% of pairs held at least one confirmed constraint, 65% at least one confirmed judgment, and 57% held both. Direct contradictions between the two documents were less common and, by our audit, much less reliably detected, so we do not use them as central evidence. The

agent and reader are rarely told opposite things. They are told different kinds of things. The mix shifts with the convention: long CLAUDE.md files carry the most project judgments, in 98.9% of their pairs, while the structured .github/instructions format carries the most explicit rules.

Additionally, the agent-facing layer is rarely announced. Across the 4,094 README-bearing repositories, only 16.7% name the instruction file by path, and including any broad mention of an agent tool or convention raises this only to 33.8%. For about two thirds of repositories, a newcomer reading the README gets no sign that an agent-facing layer exists. The files are public, but they often sit outside the README-based first-contact path we measure. The new layer also tends to accrete rather than replace: tracking READMEs around the point an instruction file appears, we find they do not shrink toward a thin overview, so the agent-facing document is added next to the human one, not swapped for it.

Three short cases give this texture. *An unseen invariant.* One instruction file marks a specific error type as an internal signal only. It instructs the agent that a helper “must strip it before returning so callers’ own backoff .Retry loops aren’t hijacked.” The file notes that this rule was added in a specific earlier commit and mandates that the agent “preserve this invariant when touching exec/select/exists.” The README never mentions it, and the resulting patch would carry no sign of the constraint that shaped it. *Erasing the agent.* A high-recall keyword scan of all 4,664 files, with manual review, found 19 distinct instruction texts (across 21 repositories, after setting aside forks and mirrors) that tell the agent to leave no trace of itself, with rules like “never add yourself to the list of commit co-authors” and “do not mention AI in commit messages.” Five repositories require the opposite, a Co-Authored-By line. The practice is uncommon, but it runs in both directions, and the structure is the same: whether an agent’s hand is recorded is settled in advance, in a file, not when a collaborator reads the commit. This is not only a code-review problem. The commit log and the review record are not only technical traces; they can become evidence in mentoring, performance review, and promotion. A rule that removes the agent from the record sets in advance whose effort will be seen and whose will not, functioning as a form of documentary control over the boundary between visible and invisible work [8]. In organizations that score AI use itself as part of performance evaluation, an instruction that hides the agent’s hand does more than shift credit: it can distort the very measure the evaluation relies on. *A gap that closes.* Where a README is itself a real architecture document, the gap nearly disappears: the same naming rules and design judgments that elsewhere live only in the agent file are also written for human readers. The split is not a law of agent files. It is a choice about where a project’s knowledge is allowed to be seen.

4 Making the Instruction Layer Reviewable

A common way to test accountability is to hand a team a finished project and its records and ask them to reconstruct who decided what, whose view won a disagreement, and who should answer if a claim turns out wrong. Instruction files put a wall one step before that exercise. Even when the instruction file sits somewhere in repository history, the record that a team is handed does not tie a contribution to the version of the instructions in force when it was

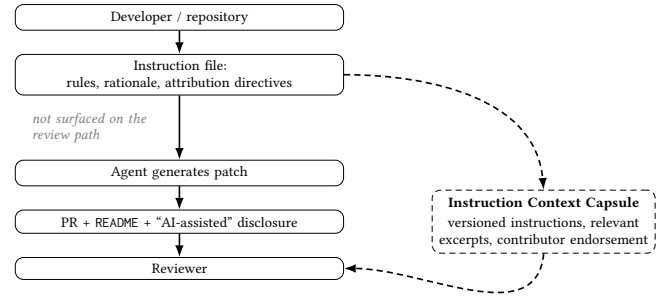


Figure 1: The instruction file is positioned to shape the patch but stays off the review path. The Capsule makes that context reviewable.

produced, nor show which of those instructions were relevant. A team could hold every commit and still not see which framing the agent was told to treat as authoritative. The exercise keeps hitting this wall until the instruction layer is part of what gets handed over.

If the terms of a contribution are set before the work, tools that act after the work arrive too late. Disclosure and provenance assume the needed evidence already exists and only has to be surfaced. Our data points to a harder case (Figure 1). Even a log of the agent’s actions would not necessarily tell a reviewer that the agent was instructed to treat one module as the source of truth or to never touch another. That context remains in the instruction file, which only about one in six READMEs points to.

We offer one design provocation, an *Instruction Context Capsule*. At pull-request time, the system would snapshot the version of the instruction artifacts positioned to guide the agent (temporal binding), surface only the rules and judgments relevant to the changed files rather than the whole file (selective surfacing), and ask the human contributor to endorse, reject, or contextualize them (human endorsement). Reviewers could then contest not only the patch but the local authority it rests on, asking why a module is treated as the source of truth, or why an invariant never reached the README (contestability). The capsule would separate personal preference from repository-governed policy and allow private context to be redacted (governance boundary). This goes beyond provenance logging. A provenance record can show what happened; the capsule also surfaces the instruction context against which the contribution can be reviewed. It turns instruction files from peripheral agent-facing artifacts that rarely enter the review path into reviewable collaborative objects, through three shifts: from off-path to review-visible, from static to versioned, and from given to contestable.

To make this concrete, picture the review screen. Beside the diff, a reviewer of the invariant case would see the one rule that governed the change, drawn from the version of the file in force at the time, and a line from the contributor saying the rule was relevant and that they endorse it as the reason for the edit. The reviewer could then ask why the invariant never reached the README, and the project could decide to move it there. Nothing about the agent’s run changes. What changes is that the basis for the contribution is on the table while the contribution is being judged, which is what turns a record into a moment of accountability.

The hardest line to draw is the governance one. A rule like "treat this module as the source of truth" is a project decision a reviewer should see. A developer's private note to the agent about how they like to think may not be. A capsule is only useful if it can hold these apart, which is a social judgment as much as a technical one, and a good reason to design it with the people whose work it would surface rather than for them. For the same reason, the capsule should not become an AI-use score or a productivity dashboard; its job is to show which rules were relevant to a change and what the contributor is willing to stand behind.

5 Discussion: From AI Use to Accountable Use

Our findings move the accountability question from whether AI was used to what would make AI use accountable. We bring three tensions to the workshop, each grounded in a finding above.

The first tension is scope. An instruction can restrict how an AI agent writes code, even if the final code shows no trace of that rule. While we know the instruction exists, it is very difficult to prove whether a specific rule actually caused a specific code change. A related problem is authorization. An AI agent may receive a developer's personal preference, a tool's default setting, and an official project policy through the same instruction channel, without a clear indication of the authority each represents. This is especially problematic in solo projects. When the same person writes the instructions and reviews the output, there is no peer review to challenge these rules.

The second tension is what review should surface. Showing every prompt and private note would create surveillance, not accountability, but a rule, invariant, or design judgment that governed a change should be available where the change is judged. The capsule's selective surfacing is one answer, though the line between relevant and private is not obvious, and personal preference must be separable from project policy when one person's habit reaches the agent as if it were shared. Instructions that erase the agent's own trace sit at the sharp end of this tension because they can undermine the disclosure record itself.

The third tension is evaluation. In organizations that score both AI use and output quality, a record of AI involvement shaped in advance leaves the evaluator without a reliable signal for either measure. The useful evidence is not how much AI was used but what the worker understood, checked, and can stand behind, and it has to be gathered without turning instruction context into surveillance.

A caution runs under all three. More visibility is not automatically more accountability. Disclosing every agent context could expose private work, proprietary information, or unfinished reasoning. The design problem is selective visibility, not total transparency.

6 Limitations

Our corpus comes from public GitHub, so it likely over-represents solo and hobby repositories and under-represents private workplace projects where AI instructions are set by a team. We read the solo result as a property of public adoption, not a claim about all software work. It still matters for collaboration because some solo repositories later become collaborative: through outside pull requests, maintainer onboarding, forks, or handoff. Contributor counts can be misleading; a seemingly solo repository may reflect

a private team, a company mirror, or a squashed-commit workflow. Private workplaces may enforce stricter instruction file policies than public repositories show. We also note that a README is a first-contact baseline, not a repository's full documentation. Most importantly, a divergence between documents does not prove that the AI followed a specific hidden instruction, nor that human developers were actually misled. Therefore, we treat our findings as an indicator of structural risk, rather than direct evidence of a breakdown in human-agent collaboration.

7 Conclusion

AI accountability usually begins after an output exists: was an agent used, and was it disclosed? These questions begin at the wrong moment. Repository-level instruction files can set the rules, judgments, and authorities under which an agent-mediated contribution will form, while those conditions stay off the path through which collaborators later review the work. Documentation divergence is not unique to agent-facing files. What changes is its accountability role: these documents are written to condition an automated contributor and can create an additional audience-specific documentation layer across the solo long tail of software. Coding instruction files are only one instance. Custom instructions, system prompts, and project guideline documents can create the same before-the-work instruction layer for agents that write, design, research, and decide. The accountability challenge extends beyond software: evaluators may face the same gap between what was produced and what can be seen of how it was formed. The contribution may be new, but the conditions of its formation were already written elsewhere. The challenge is to make these forming conditions visible where the work is judged, ensuring that a contribution can still be credited and questioned rather than left unaccountable.

References

- [1] Anthropic. 2026. *2026 agentic coding trends report*. Technical Report. Anthropic. <https://resources.anthropic.com>
- [2] Alberto Bacchelli and Christian Bird. 2013. Expectations, outcomes, and challenges of modern code review. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. IEEE, 712–721.
- [3] Worawalan Chatlatanagulchai, Hao Li, Yutaro Kashiwa, Brittany Reid, Kundjanasith Thonglek, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, Bram Adams, Ahmed E. Hassan, and Hajimu Iida. 2025. Agent READMEs: An empirical study of context files for agentic coding. *arXiv preprint arXiv:2511.12884* (2025).
- [4] Worawalan Chatlatanagulchai, Kundjanasith Thonglek, Brittany Reid, Yutaro Kashiwa, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, and Hajimu Iida. 2025. On the use of agentic coding manifests: An empirical study of Claude Code. In *Product-Focused Software Process Improvement (PROFES)*. arXiv:2509.14744.
- [5] Kashif Imteyaz, Mohammad Rashidujjaman Rifat, Divya Ramesh, Steven R. Rick, Simo Hosio, Hauke Sandhaus, Advait Sarkar, Christoph Riedl, and Saiph Savage. 2026. "Nobody Did This": Contribution, Originality, and Accountability in Agent-Mediated Collaboration. Workshop at the ACM Conference on Computer-Supported Cooperative Work and Social Computing (CSCW '26). arXiv:2607.26387 [cs.CY] doi:10.48550/arXiv.2607.26387
- [6] Jai Lal Lulla, Seyedmoein Mohsenimofidi, Matthias Galster, Jie M. Zhang, Sebastian Baltes, and Christoph Treude. 2026. On the impact of AGENTS.md files on the efficiency of AI coding agents. In *Journal Ahead Workshop (JAWs) at ICSE*. arXiv:2601.20404.
- [7] K. Schmidt and L. Bannon. 1992. Taking CSCW seriously: Supporting articulation work. *Computer Supported Cooperative Work* 1, 1 (1992), 7–40.
- [8] S. L. Star and A. Strauss. 1999. Layers of silence, arenas of voice: The ecology of visible and invisible work. *Computer Supported Cooperative Work* 8, 1 (1999), 9–30.
- [9] A. Strauss. 1985. Work and the division of labor. *The Sociological Quarterly* 26, 1 (1985), 1–19.

- [10] L. A. Suchman. 1987. *Plans and Situated Actions: The Problem of Human-Machine Communication*. Cambridge University Press.